

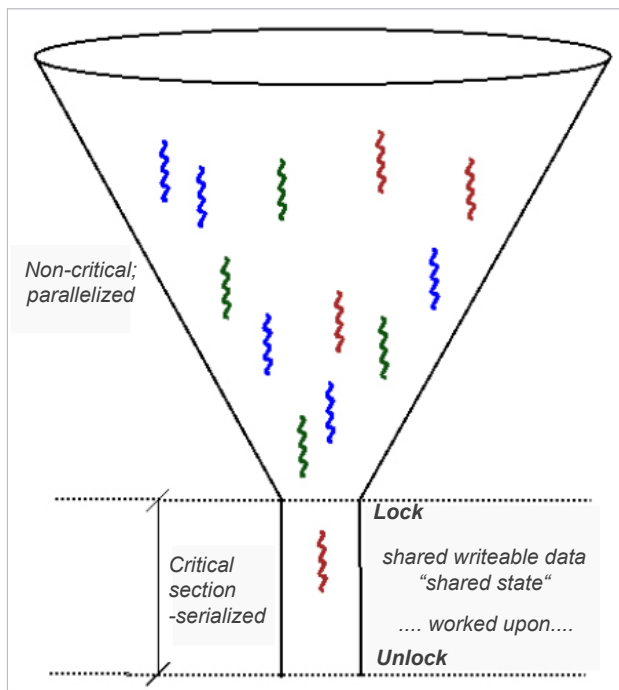


Figure 12.4 – A lock creates a bottleneck, analogous to a physical funnel

- When running in an atomic (interrupt) non-blocking context– atomically: indivisibly, to completion, without interruption. Once protected, you can safely access your shared state until you “unlock”.
- Every critical section in the code base must be identified and protected:
 - Identifying critical sections is critical! Carefully review your code and make sure you don’t miss them.
 - Protecting them can be achieved via various technologies; one very common technique is *locking* (there’s also lock- free programming, which we’ll look at in the next chapter)
 - A common mistake is only protecting critical sections that write to global writeable data; you must also protect critical sections that read global writeable data; otherwise, you risk a torn or dirty read! To help make this key point clear, visualize an unsigned 64-bit data item being read and written on a 32-bit system; in such a case, the operation can’t be atomic (two load/store operations are required). Thus, what if, while you’re reading the value of the data item in one thread, it’s being simultaneously written to by another thread!? The writer thread takes a “lock” of some sort but because you thought reading is safe, the lock isn’t taken by the reader thread; due to an unfortunate timing coincidence, you can end up performing a partial/torn/dirty read! We will learn how to overcome these issues by using various techniques in the coming sections and the next chapter.

- Another deadly mistake is not using the same lock to protect a given data item.
- Failing to protect critical sections leads to a data race, a situation where the outcome – the actual value of the data being read/written – is “racy”, which means it varies, depending on runtime circumstances and timing. This is known as a bug. (A bug that, once in “the field”, is extremely difficult to see, reproduce, determine its root cause, and fix. We will cover some very powerful stuff to help you with this in the next chapter, in the *Lock ebugging within the kernel* section; be sure to read it!)
- **Exceptions:** You are safe (implicitly, without explicit protection) in the following situations:
 - When you are working on local variables. They’re allocated on the private stack of the thread (or, in the interrupt context, on the local IRQ stack) and are thus, by definition, safe.
 - When you are working on shared writeable data in code that cannot possibly run in another context; that is, it’s serialized by nature. In our context, the *init* and *cleanup* methods of an LKM qualify (they run exactly once, serially, on insmod and rmmod only).
 - When you are working on shared data that is truly constant and read-only (don’t let C’s *const* keyword fool you, though!).
- Locking is inherently complex; you must carefully think, design, and implement this to avoid deadlocks. We’ll cover this in more detail in the *Locking guidelines and deadlocks* section. Further on in the book, we shall show you how exactly you can check in which context your kernel code is currently running. Read on!



Note: The book’s source repositories are publicly available on GitHub, as indicated below.

- Linux Kernel Programming (Part 1): A comprehensive guide to kernel internals, writing kernel modules, and kernel synchronization; <https://github.com/PacktPublishing/Linux-Kernel-Programming>
- Linux Kernel Programming (Part 2): Learn to work with ‘misc’ class char drivers, user-kernel interfaces, manage peripheral I/O and hardware interrupts; <https://github.com/PacktPublishing/Linux-Kernel-Programming-Part-2>

END

By: Kaiwan N. Billimoria

The author is a founder at kaiwanTECH (Designer Graphix), and has spent long days and nights in compiling and writing this book.